

Отчёт по лабораторной работе №6

**Математические основы защиты информации и информационной
безопасности**

Ли Хан

Содержание

1	Цель работы	5
2	Модуль реализации алгоритма	6
2.1	Описание ρ -метода Полларда	6
2.2	Шаги реализации:	6
2.2.1	ρ -метода Полларда компилятора	7
2.2.2	ρ -метода Полларда компилятора	8
2.2.3	результат	9
3	Вывод	10

Список иллюстраций

2.1	ρ -метода Полларда компилятора	7
2.2	ρ -метода Полларда компилятора	8
2.3	ρ -метода Полларда результат	9

Список таблиц

1 Цель работы

Изучить основные задачи и алгоритмы разложения чисел на множители. Реализовать программно ρ -метод Полларда, понять принципы поиска нетривиальных делителей составных чисел в криптоанализе и выполнить численные расчеты на основе заданных начальных параметров.

2 Модуль реализации алгоритма

2.1 Описание ρ -метода Полларда

ρ -метод Полларда — это рандомизированный алгоритм факторизации целых чисел. Он основан на математическом принципе «парадокса дней рождения» и использует сжимающие свойства случайных отображений для поиска циклов в последовательностях. Этот алгоритм крайне эффективен для разложения чисел, имеющих небольшие простые делители, и имеет временную сложность около $O(n^{1/4})$.

2.2 Шаги реализации:

- Инициализация: Установить переменные a и b , присвоив им начальное значение c .
- Итерация: Обновлять последовательность с помощью функции $f(x) = x^2 + \text{const} \pmod{n}$. При этом a обновляется один раз за шаг ($a \leftarrow f(a)$), а b — дважды ($b \leftarrow f(f(b))$).
- Вычисление НОД: На каждом шаге вычислять $d = \text{НОД}(|a - b|, n)$.
- Проверка результата:
 - Если $1 < d < n$, то d является нетривиальным делителем числа n , алгоритм завершен успешно.

- Если $d = n$, результат: «Делитель не найден», алгоритм завершен неудачей.
- Если $d = 1$, вернуться к шагу 2 для продолжения итерации.

2.2.1 ρ -метода Полларда компилятора

```
import math

def gcd(a, b):
    """
    计算最大公约数 (НОД)。
    """
    while b:
        a, b = b, a % b
    return a

def pollard_rho_algorithm(n, c, f_const):
    """
    实现波拉德 rho 方法。
    Реализация р-метода Полларда.
    """
    # 压缩函数 f(x) = x^2 + f_const (mod n)
    # ФУНКЦИЯ f(x) = x^2 + const (mod n)
    def f(x):
        return (x**2 + f_const) % n

    # 1. 初始化 a 和 b
    # 1. Положить a <- c, b <- c
    a = c
    b = c
    d = 1

    # 打印计算过程表头
    # Печать заголовка таблицы вычислений
    print(f"{'i':<5} {'a (f(a))':<15} {'b (f(f(b)))':<15} {'d = НОД(|a-b|, n)':<15}")
    print("-" * 65)
```

Рис. 2.1: ρ -метода Полларда компилятора

2.2.2 ρ -метода Полларда компилятора

```
i = 0
# 循环执行直到找到因子或失败
# Повторять шаги до нахождения делителя
while d == 1:
    i += 1

    # 2. 更新序列值
    # 2. В ы ч и с л и т ь  a <- f(a)(mod n), b <- f(f(b))(mod n)
    a = f(a)
    b = f(f(b))

    # 3. 计算最大公约数
    # 3. Н а й т и  d <- Н О Д (a - b, n)
    d = gcd(abs(a - b), n)

    # 打印当前步的中间数据
    # П е ч а т ь  п р о м е ж у т о ч н ы х  д а н н ы х  т е к у
    print(f"{i:<5}  {a:<15}  {b:<15}  {d:<15}")

    # 4. 判断结果
    # 4. П р о в е р к а  р е з у л ь т а т а
    if d == n:
        return "Делитель не найден"
    if d > 1:
        return d
```

Рис. 2.2: ρ -метода Полларда компилятора

2.2.3 результат

```
# 样例：使用实验报告中的数据 n = 1359331, c = 1, f_const = 5
# Пример: n = 1359331, c = 1, f_const = 5
n_test = 1359331
c_test = 1
const_test = 5

print(f"Разложение числа n = {n_test}:\n")
divisor = pollard_rho_algorithm(n_test, c_test, const_test)
print("-" * 65)
print(f"Результат: найден делитель p = {divisor}")
```

... Разложение числа n = 1359331:

i	a (f(a))	b (f(f(b)))	d = НОД(a-b , n)
1	6	41	1
2	41	123939	1
3	1686	391594	1
4	123939	438157	1
5	435426	582738	1
6	391594	1144026	1
7	1090062	885749	1181

Результат: найден делитель p = 1181

Рис. 2.3: ρ -метода Полларда результат

3 Вывод

В ходе выполнения данной работы был успешно реализован и протестирован ρ -метод Полларда. При заданных параметрах $c = 1$ и константе 5 было произведено разложение числа 1359331. На 7-й итерации был найден делитель 1181. Это подтверждает эффективность и быструю сходимость данного алгоритма при поиске делителей больших составных чисел.